

PLATEFORME SAAS MULTI-ENTREPRISES & VISITE VIRTUELLE 3D

Rapport d'avancement complet du projet — Projet Hergla Park

Domaine : Plateforme Web SaaS Multi-Tenant, Supervision ROOT, Expérience 3D & VR Interactive

État du Projet : Socle Backend, Dashboard Admin, Site Vitrine VR, Rôles & Karts 100% Fonctionnels

Étape en cours : Synthèse d'architecture avant implémentation du prototype Karting Three.js

Équipe de Développement : Équipe Projet Hergla Park

Environnement d'Hébergement : Vercel (Frontend & Serverless Backend) × Neon (PostgreSQL Serverless)

2. Sommaire

1. Page de garde
2. Sommaire
3. Introduction
4. Architecture Globale Actuelle
5. Le Backend (NestJS, Prisma, Sécurité)
6. Les Interfaces (Dashboard React, Site Vitrine VR, Éditeur 3D)
7. Évolution vers le SaaS Multi-Entreprises
8. Déploiement et Infrastructure (Vercel, Neon, Monorepo Git)
9. Configuration des Karts
10. Prochaines Étapes (Prototype Karting Three.js & Unity)
11. Synthèse des Technologies Utilisées
12. Défis Rencontrés et Solutions Apportées
13. Conclusion

3. Introduction

3.1 Contexte et Vision du Projet

Le projet **Hergla Park** est né de la volonté de moderniser la gestion et la promotion d'un parc d'attractions et de karting de référence. Initialement conçu comme une application web de gestion mono-site dédiée à un unique parc, le projet a connu une trajectoire ambitieuse en se transformant en une **plateforme SaaS multi-entreprises complète**, capable d'accueillir et d'isoler plusieurs organisations, d'offrir un contrôle d'accès granulaire et de proposer des expériences immersives en 3D et Réalité Virtuelle.

3.2 Portée du Rapport

Ce document constitue un audit technique rigoureux et un rapport d'avancement complet à l'attention de l'encadrement académique et du jury. Il dresse l'état exact des fonctionnalités développées, de l'infrastructure déployée, des choix d'architecture et des évolutions stratégiques effectuées à ce jour. Chaque affirmation contenue dans ce rapport s'appuie sur le **code source réel et vérifié** du dépôt (dossiers `backend/`, `dashboard/`, `vr-landing/`, `3d-assets/`, `docs/`).

Résumé de la Trajectoire du Projet : En l'espace de deux semaines de développement intensif, le système est passé d'un simple CRUD de gestion d'un parc local à une architecture SaaS multi-tenant hébergée sur le cloud serverless (Vercel + Neon PostgreSQL), dotée d'un rôle d'administration globale ROOT avec journalisation renforcée, d'un Dashboard d'administration React bilingue (Français/Arabe avec RTL), d'un site vitrine interactif VR et d'un configurateur 3D de karts et d'espaces sous React Three Fiber.

4. Architecture Globale Actuelle

L'architecture globale est articulée autour d'un monorepo structuré garantissant une séparation nette des responsabilités, une sécurité de haut niveau et une réutilisation optimale des composants.

Composant	Rôle Principal	Technologies Clés	Mode d'Interconnexion
Backend REST API	Logique métier, Authentification, Autorisations, Supervision ROOT, Karts	NestJS, TypeScript, Prisma ORM, JWT, Swagger	HTTPS / REST JSON via CORS sécurisé
Base de Données	Stockage relationnel multi-tenant, rôles, karts, agencements 3D, audit logs	Neon (PostgreSQL Serverless)	Connection Pooling (Transaction) & Connection Directe (DirectURL)
Dashboard Admin	Gestion des espaces, utilisateurs, rôles, configuration karts, éditeur 3D	React 19, Vite, TailwindCSS v4, i18next (FR/AR), R3F	API REST Authentifiée (Bearer JWT Token)
Site Vitrine VR	Parcours utilisateur 4 étapes, règles de sécurité, lancement expérience VR	React 19, Framer Motion, i18next, Lucide Icons	Endpoint Public REST minimaliste (Karts publics)
Infrastructure Cloud	Hébergement continu, fonctions serverless, prévisualisation par branche	Vercel, Git LFS, GitHub Actions	Intégration native Vercel × Neon

5. Le Backend

5.1 Qu'est-ce qui a été construit ?

Le backend est l'élément central du système. Il fournit une API REST robuste, typée et entièrement documentée. Il gère le cycle de vie des utilisateurs, des espaces d'attraction, des objets 3D, de la configuration des karts, des rôles dynamiques et du journal d'audit centralisé.

5.2 Avec quelle technologie, et pourquoi ce choix ?

- **NestJS (TypeScript)** : Choisi pour son architecture modulaire orientée objets (inspirée d'Angular), son système d'injection de dépendances et sa conformité aux standards d'entreprises. Il garantit la maintenabilité du code lors de la montée en charge.
- **Prisma ORM** : Sélectionné à la place d'ORM traditionnels (ex: TypeORM) pour sa sécurité de typage automatique de bout en bout (Type-Safety) générée directement à partir du schéma de base de données, ainsi que pour la simplicité de ses migrations.
- **Authentification JWT + Bcrypt** : Implémentation de jetons JSON Web Tokens pour une authentification stateless parfaitement adaptée au modèle Serverless, combinée avec le hachage sécurisé Bcrypt (salage fort) des mots de passe.

5.3 Comment ça fonctionne concrètement ?

Chaque requête entrante passe par une série de filtres et de gardes (Guards NestJS) :

1. `JwtAuthGuard` : Vérifie la validité du jeton JWT présent dans l'en-tête HTTP `Authorization: Bearer <token>`.
2. `RolesGuard` & `PermissionsGuard` : Vérifient que l'utilisateur possède les rôles nécessaires ou les clés de permission spécifiques (ex: `espace:create`, `scene:edit`, `role:manage`).
3. `RootGuard` : Valide que l'utilisateur actif est titulaire du rôle système ROOT (niveau 100) pour les actions d'infrastructure ou de supervision globale.

La documentation interactive de l'API est automatiquement générée via OpenAPI et consultable à l'adresse `/api/docs`.

6. Les Interfaces Web

6.1 Dashboard Administration React

Développé avec React 19 et Vite, le Dashboard d'administration constitue le cockpit opérationnel pour les administrateurs de parcs et le rôle ROOT.

- **Internationalisation & RTL** : Support bilingue intégral Français / Arabe via `i18next`. L'interface bascule dynamiquement l'orientation globale du document (`dir="rtl"`) pour offrir une expérience arabe native.
- **Gestion des Espaces (/espaces)** : Tableau de bord synthétique affichant le statut opérationnel (OUVERT, FERME, MAINTENANCE), avec filtrage par catégorie et modification rapide par modale.

- **Gestion des Utilisateurs (`/users`)** : Attribution de rôles, filtrage par entreprise et gestion des accès.

6.2 Site Vitrine "Visite Virtuelle" (VR Landing)

Une application front-end dédiée aux visiteurs du parc, conçue pour préparer l'expérience immersive en 4 étapes fluides :

1. **Bienvenue (`WelcomePage`)** : Présentation immersive du parc Hergla Park et vidéo d'introduction.
2. **Contrôles (`ControlsPage`)** : Guide interactif des commandes de conduite du kart et manettes VR.
3. **Sécurité (`SafetyPage`)** : Consignes d'urgence, règles de sécurité en piste et drapeau de course.
4. **Lancement (`LaunchPage`)** : Sélection de l'espace de karting et récupération en direct des karts configurés via l'endpoint public minimal du backend.

6.3 Éditeur de Scène 3D (React Three Fiber)

Intégré directement dans le Dashboard (`/scene-editor`), cet outil permet aux responsables d'agencer visuellement les objets 3D dans un espace :

- **Catalogue & Glisser-Déposer** : Consultation des modèles 3D (mobilier, signalétique, décorations) au format `.glb`.
- **Transformation 3D** : Modification en temps réel des coordonnées XYZ (Position, Rotation, Échelle).
- **Sauvegarde & Reset** : Enregistrement des placements dans la table `ScenePlacement` du backend et possibilité de réinitialiser la scène à l'état d'origine (snapshot `originalSceneData`).

7. Évolution vers la Plateforme SaaS Multi-Entreprises

Le passage d'une application mono-parc à une plateforme multi-tenant a exigé une restructuration profonde du modèle de données et des mécanismes de contrôle d'accès.

7.1 Isolation Multi-Tenant

Chaque donnée appartient désormais à une entreprise spécifique identifiée par le modèle `Company` (nom, slug unique, logo). Les modèles `User`, `Espace`, `Object3D`, `Role` et `AuditLog` intègrent obligatoirement la clé étrangère `companyId`.

7.2 Migration de la Base de Données vers Neon (PostgreSQL Serverless)

Initialement basé sur une instance locale/Docker, le projet a migré vers **Neon**, la plateforme PostgreSQL cloud serverless.

- **Avantages** : Haute disponibilité, séparation du stockage et du compute, mise à l'échelle automatique.
- **Database Branching** : Neon permet de créer des branches virtuelles de la base de données de manière instantanée, s'intégrant parfaitement avec les prévisualisations Vercel.
- **Gestion des Connexions** : Utilisation de `DATABASE_URL` (pooling via PgBouncer pour les fonctions serverless) et `DIRECT_URL` (connexion directe pour les migrations Prisma).

7.3 Système de Rôles Dynamique et Hiérarchie

Au lieu de rôles statiques codés en dur, la plateforme intègre un système granulaire basé sur `Role`, `Permission` et `UserRole`.

- **Niveau de Hiérarchie (champ `niveau`)** :
 - `100` : ROOT (Supervision plateforme)
 - `90` : SUPERADMIN (Gestionnaire multi-sites / groupe)
 - `50` : ADMIN (Gestionnaire d'un parc spécifique)
 - `20` : EMPLOYE / Rôle personnalisé
- **Règle d'attribution stricte** : Un utilisateur possédant un rôle supérieur peut attribuer des rôles de niveau inférieur en complément. Toutefois, pour des raisons de sécurité, l'attribution d'un rôle supérieur ou la modification fondamentale du rôle nécessite une action explicite (`mode: replace`).

7.4 Rôle ROOT & Supervision Transverse

Le rôle **ROOT** dispose d'un droit de regard et d'action transverse sur l'ensemble de la plateforme :

- Consultation du journal d'activité global (`AuditLog`).
- Création et gestion des entreprises clientes.
- **Intervention d'Urgence** : Capacité d'agir directement sur les données d'une entreprise en cas de panne ou de support d'urgence. Toute action effectuée par un compte ROOT enregistre automatiquement le flag `isRootIntervention = true` et exige un motif explicite dans les métadonnées pour une traçabilité totale.

7.5 SUPERADMIN Multi-Entreprises

Grâce à la table de liaison `UserCompany` et à l'endpoint `POST /auth/switch-company`, un même compte utilisateur SUPERADMIN peut être rattaché à plusieurs entreprises d'un même groupe et basculer instantanément de contexte de travail sans devoir se déconnecter.

8. Déploiement et Infrastructure

8.1 Hébergement Vercel x Neon

L'ensemble de l'écosystème est déployé sur l'infrastructure Cloud moderne :

- **Backend NestJS** : Adapté sous forme de fonctions Serverless Vercel (fichier `api/index.ts` wrapper).
- **Dashboard & VR Landing** : Compilés et distribués sur le réseau CDN mondial de Vercel avec rechargement instantané.
- **Intégration Native** : Chaque Pull Request GitHub génère une URL de prévisualisation unique couplée à une branche de base de données Neon dédiée.

8.2 Arborescence du Monorepo Git & Git LFS

Le dépôt GitHub a été réorganisé de manière propre et isolée :

- `backend/` : Code source NestJS & schéma Prisma.
- `dashboard/` : Application React d'administration.
- `vr-landing/` : Site vitrine interactif VR.
- `3d-assets/` : Stockage des modèles 3D volumineux (fichiers `.glb`) gérés via **Git LFS (Large File Storage)**.
- `docs/` : Documentation technique, checklists QA et rapports PDF.

8.3 Préparation aux Tests & Qualité (QA)

Afin de garantir la fiabilité du système avant les présentations académiques :

- **Jeu de Données de Démonstration (Seed)** : Script automatisé (`prisma/seed.ts`) initialisant les entreprises, utilisateurs de test (ROOT, SUPERADMIN, ADMIN, EMPLOYE) et espaces de démonstration (documentés dans `SEED_CREDENTIALS.md`).
- **Checklist QA Structurée (`docs/CHECKLIST_QA.md`)** : Cahier de recettes répertoriant les scénarios de test par rôle.
- **CORS & Security** : Validation de la politique d'origine croisée pour autoriser les requêtes entre les différents domaines Vercel et le backend.

9. Configuration des Karts

9.1 Modèle de Données `Kart`

Chaque karting est modélisé dans la base de données relationnelle et rattaché à un espace d'attraction :

- `espaceId` : Identifiant de l'espace de karting associé.
- `numero` : Numéro de course (ex: "07", "12") affiché sur le véhicule. Unicité garantie par espace (`@@unique([espaceId, numero])`).
- `couleur` : Code couleur hexadécimal (ex: `#E53935`) pour la carrosserie.
- `actif` : Statut d'activation (permet de désactiver un kart en maintenance sans supprimer ses données historiques).

9.2 Panneau Dedicated Dashboard & Aperçu 3D Temps Réel

Dans le Dashboard (`/karts-config`), les administrateurs disposent d'un panneau intuitif permettant de :

1. Définir la flotte de karts par espace.
2. Ajuster la couleur via un sélecteur dynamique.
3. Visualiser le rendu final en direct grâce au composant `KartPreviewCanvas.jsx` utilisant React Three Fiber. Le modèle 3D du kart réagit instantanément aux changements de couleur avant toute validation en base.

Un endpoint public minimaliste (`GET /karts/espace/:espaceId/public`) permet au Site Vitrine VR d'afficher directement les karts disponibles aux visiteurs sans nécessiter d'authentification.

10. Prochaines Étapes : Prototype Karting 3D & Roadmap

10.1 État Réel au Moment du Rapport

Statut du Prototype Karting Three.js : **PLANIFIÉ / PROCHAINE ÉTAPE**

L'analyse technique et les dépendances 3D (Three.js et React Three Fiber) sont en place dans le projet (utilisées pour l'éditeur de scène et l'aperçu 3D des karts). Le développement spécifique du prototype de conduite avec moteur physique (Rapier) constitue la toute prochaine étape de développement.

10.2 Analyse Stratégique : Approche Hybride Three.js + Unity

Après étude comparative, l'équipe a validé une **approche hybride** :

Critère	Prototype Web Three.js + Rapier	Application Immersive Unity WebXR
Objectif	Validation ultra-rapide du gameplay et de la conduite sur navigateur	Expérience VR totale avec casque (Meta Quest / HTC Vive)
Temps de Développement	Court (Immédiat dans l'écosystème React/Vite existant)	Moyen / Long (Semaines 5 à 8 du planning)
Accessibilité	100% Web (Zéro installation, fluide sur PC & Mobile)	Casque VR / Navigateur compatible WebXR
Moteur Physique	Rapier.js (Physique 3D Rust compilée en WebAssembly)	Unity PhysX Engine (Simulations complexes)

10.3 Planning Prévisionnel (Semaines 3 à 8)

- **Semaines 3 - 4** : Finalisation du prototype karting Three.js avec physique Rapier (gestion des collisions, accélération, dérapage) et modélisation détaillée de la piste sous Blender.
- **Semaines 5 - 6** : Intégration de l'application Unity WebXR, importation des assets 3D optimisés et liaison API avec le backend NestJS.
- **Semaines 7 - 8** : Tests de performance (garantie 60 FPS constants en VR), ajustements UX et déploiement final de l'ensemble de la plateforme.

11. Synthèse des Technologies Utilisées

Couche	Technologie	Usage Exact dans le Projet	Statut Code
Backend Framework	NestJS (v10)	Architecture API REST, Services, Contrôleurs, Guards de Sécurité	Implémenté
ORM & Database	Prisma ORM & Neon PostgreSQL	Modélisation relationnelle multi-tenant, migrations, connection pooling	Implémenté
Sécurité & Auth	JWT & Bcrypt	Jetons d'authentification stateless et hachage sécurisé des passwords	Implémenté
Frontend Admin	React 19 & Vite	Single Page Application pour l'administration et le contrôle des parcs	Implémenté
Design & Styling	TailwindCSS v4 & Lucide Icons	Composants d'interface modernes, réactifs et thématiques	Implémenté
Internationalisation	i18next & react-i18next	Gestion du bilinguisme Français / Arabe avec support dynamique RTL	Implémenté
Site Vitrine VR	React 19 & Framer Motion	Parcours utilisateur interactif 4 étapes et animations fluides	Implémenté
Rendu & Édition 3D	Three.js & React Three Fiber	Éditeur de scène 3D (catalogue/placement) & Aperçu 3D des karts	Implémenté
Physique Karting	Rapier.js / @react-three/rapier	Simulation physique de la conduite du kart en navigateur	Planifié
Hébergement Cloud	Vercel Cloud Platform	Déploiement continu frontend et exécution des fonctions serverless backend	Implémenté

12. Défis Rencontrés et Solutions Apportées

12.1 Défis d'Architecture et de Base de Données

- Problème** : Les environnements serverless (Vercel) ouvrent une nouvelle connexion à la base de données à chaque requête, risquant de saturer le nombre maximal de connexions PostgreSQL.
- Solution** : Configuration du Connection Pooling Neon via PgBouncer avec le paramètre DATABASE_URL pour les requêtes de données, et conservation d'une DIRECT_URL non poolée réservée exclusivement aux CLI de migrations Prisma.

12.2 Défis de Migration SaaS Multi-Tenant

- **Problème** : Transformer un modèle mono-entreprise existant en multi-tenant sans perdre la cohérence des données ni casser les fonctionnalités en place.
- **Solution** : Introduction d'une migration Prisma progressive avec valeur par défaut temporaire, ajout de la table `Company` et extension des gardes d'authentification pour injecter le contexte d'entreprise dans chaque requête HTTP.

12.3 Restructuration du Dépôt et Assets Volumineux

- **Problème** : L'ajout de modèles 3D (fichiers `.glb` lourds) risquait de ralentir considérablement les opérations Git et de dépasser les limites de taille de dépôt GitHub.
- **Solution** : Mise en place de **Git LFS (Large File Storage)** pour suivre spécifiquement les extensions d'assets 3D, associée à une restructuration propre en Monorepo et une sauvegarde de sécurité préalable.

13. Conclusion

À cette étape de l'avancement, le projet **Hergla Park SaaS** repose sur des bases extrêmement solides, modernes et évolutives. L'ensemble du socle backend, la gestion avancée des rôles et des autorisations, l'isolation multi-tenant, la supervision ROOT, le Dashboard admin bilingue et le configurateur 3D des karts et des scènes sont **pleinement opérationnels et déployés en production**.

L'analyse technique rigoureuse et la clarté des choix d'ingénierie présentées dans ce rapport démontrent la maturité de l'architecture. L'équipe est désormais dans des conditions idéales pour aborder la phase suivante : l'implémentation du prototype de simulation karting Three.js avec moteur physique, suivie de l'intégration VR Unity, conformément aux objectifs académiques et professionnels fixés.